



MEMORY MANAGEMENT

How Spark Splits Executor RAM Between Execution & Storage

Unified Memory

Execution

Storage

Spill

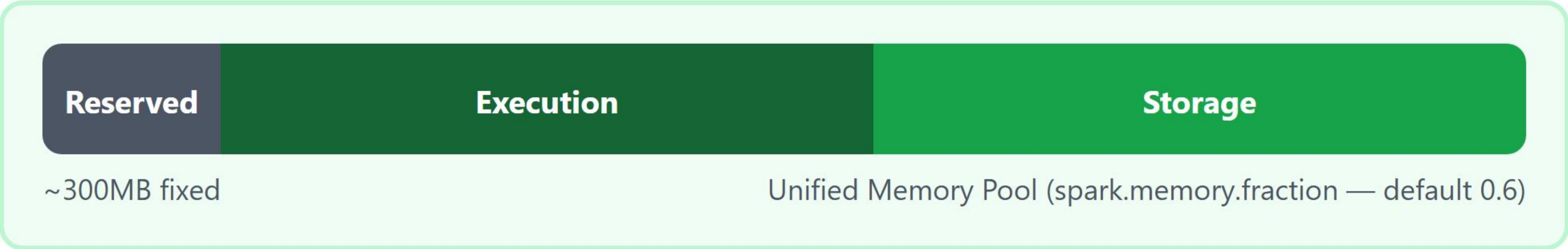
Interview Ready in 6 Slides

Swipe to start learning →

M UNIFIED MEMORY MANAGEMENT

Since Spark 1.6, **Execution** and **Storage** share one region of Executor heap instead of two fixed, walled-off pools.

Splitting the JVM Heap



- 1 **Reserved Memory** — ~300MB hard-coded, protects Spark's internals from OOM
- ↓
- 2 **spark.memory.fraction** (default 0.6) carves out the Unified Memory pool
- ↓
- 3 That pool splits again into **Execution** + **Storage** Memory

Interview Tip: `spark.memory.storageFraction` (default 0.5) sets only the *initial* boundary — the two sides can still borrow from each other.

E TWO MEMORY, ONE POOL

Execution Memory

Powers shuffles, joins, sorts, and aggregations — needed to actually run a Task.

Storage Memory

Holds cached/persisted RDDs and DataFrames via **.cache()** / **.persist()**.

Who Wins the Tug-of-War

1

If Execution needs more room, it can **evict cached blocks** from Storage



2

Storage can **never** evict blocks that Execution is actively using



3

This asymmetry stops a running Task from being starved by cached data

Interview Tip: Execution memory always wins the tug-of-war — a deliberate design choice to keep Jobs from failing over cached data.

S SPILL — MEMORY OVERFLOW TO DISK

When a Task's working set (a big shuffle sort or aggregation) can't fit in its share of Execution Memory, Spark **spills** the excess to local disk.

The Spill Sequence

1

Task tries to hold all its shuffle data in memory



2

Memory pressure crosses the threshold → extra data serializes to disk



3

Spill files are merged back in during the read phase (external sort-merge)



4

The Job still finishes — but now bottlenecked by disk I/O, not RAM speed

Interview Tip: Watch "Shuffle Spill (Memory)" and "Shuffle Spill (Disk)" in the Spark UI — non-zero values mean you're memory-starved.

T PRACTICAL MEMORY TUNING

Most memory problems are fixed with a handful of levers — not a full rewrite.

Four Levers Worth Knowing

1

Raise **spark.executor.memory** — but watch for long GC pauses on very large heaps



2

Lower **spark.memory.storageFraction** for compute-heavy jobs with little caching



3

Call **.unpersist()** on cached DataFrames once they're no longer needed



4

Prefer Kryo / **spark.memory.offHeap.enabled** to cut garbage-collection overhead

Interview Tip: Forgotten `.cache()` calls are one of the most common causes of executor OOMs in real pipelines.

Save this slide before your next Data Engineering interview

REGION	PURPOSE	KEY INTERVIEW FACT
Reserved Memory	Protects Spark internals	~300MB, fixed
Execution Memory	Shuffles, joins, sorts	Can evict Storage memory
Storage Memory	Cached RDDs/DataFrames	Evictable, never blocks Execution
Spill	Overflow to disk	Signals memory pressure, slows jobs

Follow for daily Data Engineering content

@chintapradeep

Spark · Python · SQL · Interview Prep